

I 1 Kinh tế học của pretraining (The Economics of Pretraining)

PHỤ THUỘC C/18/01-nguồn-giam-sat (mục ngay trước, bắt buộc); B/11-gioi-han-tinh-toan (compute, băng thông, FLOPs); B/07-chia-du-lieu-va-danh-gia (vì sao nhiệm vụ tập kiểm tra phá huỷ mọi kết luận); E/24-thi-nghiem-va-ablation (cách chạy thí nghiệm điểm giao cho từ từ).

TÓM TẮT

Mục này là mặt định lượng của việc tiền huấn luyện: có một ngân sách compute cố định thì chia cho tham số và dữ liệu theo tỉ lệ nào, và khi nào tính chính đáng giá hơn đồng bằng. Đọc xong bạn ước lượng được chi phí một lần tiền huấn luyện bằng một phép nhân trên giấy. Bạn đọc được một scaling law mà không bị nó lừa. Và bạn thiết kế được thí nghiệm điểm giao cho chính dữ liệu của mình, thay vì tin lời khuyên chung. Nếu chỉ nhớ một điều: compute, tham số và dữ liệu là ba mặt của cùng một ngân sách — bài học Chinchilla là ví dụ đắt nhất lịch sử ngành về việc tối ưu một mặt mà quên hai mặt kia.

KIẾN THỨC CẦN CÓ TRƯỚC

- **FLOPs và ngân sách compute** — FLOPs là số phép tính dấu phẩy động. Ngân sách compute C là tổng FLOPs bạn mua được cho một lần huấn luyện; nó cố định, nên chọn mô hình to hơn đồng nghĩa với chọn dữ liệu ít hơn.
- **token** — đơn vị nhỏ nhất của dữ liệu huấn luyện mà mô hình xử lý — một mẫu từ với văn bản, một mảnh ảnh (*patch*) với ViT. Lượng dữ liệu D luôn được đếm bằng token, không phải bằng số file.
- **hàm mất mát (loss)** — con số đo mức sai của mô hình trên dữ liệu; huấn luyện là quá trình giảm nó. Mọi “scaling law” trong mục này là công thức dự đoán con số đó sẽ giảm tới đâu.
- **scaling law** — quan hệ định lượng khớp từ thực nghiệm giữa loss, số tham số, lượng dữ liệu và compute. Giá trị của nó là cho phép *ngoại suy*: chạy vài mô hình nhỏ rồi dự đoán mô hình lớn trước khi tiêu tiền.
- **tập kiểm tra (test set) và tập validation** — tập validation dùng để chọn siêu tham số; tập test dùng đúng một lần để báo cáo. Trộn hai vai trò này là cách phổ biến nhất để tự lừa mình.
- **siêu tham số (hyperparameter)** — tham số bạn chọn chứ không phải mô hình học, ví dụ *learning rate* (độ dài mỗi bước cập nhật trọng số), kích thước *batch*, số *epoch*.
- **checkpoint** — ảnh chụp toàn bộ trọng số mô hình (và thường cả trạng thái optimizer) tại một thời điểm huấn luyện, lưu ra đĩa để chạy tiếp hoặc để triển khai.
- **seed** — số khởi tạo bộ sinh ngẫu nhiên. Đổi seed là đổi khởi tạo trọng số, thứ tự dữ liệu và augmentation — nên hai lần chạy khác seed cho hai kết quả khác nhau ngay cả khi mã hoàn toàn giống nhau.

A. Đặt vấn đề

Mục trước trả lời câu hỏi *lấy giám sát ở đâu*. Mục này trả lời câu hỏi đắt tiền hơn: *có một ngân sách compute cố định thì tiêu vào đâu?* Đây không phải câu hỏi triết học — nó là câu hỏi phân bổ nguồn lực, và trong nhiều năm cả ngành đã trả lời sai nó theo cùng một hướng.

Vấn đề khó ở chỗ ba đại lượng — số tham số N , lượng dữ liệu D , và compute C — ràng buộc nhau. Thêm tham số mà không thêm dữ liệu thì mô hình có thêm chỗ chứa mà không có gì để chứa. Thêm dữ liệu mà không thêm tham số thì mô hình bão hoà. Và cả hai đều bị chặn bởi tổng số phép tính bạn mua được. Nếu chỉ có trực giác, cách duy nhất tìm điểm cân bằng là thử; nhưng mỗi lần thử ở quy mô nghìn tỉ tham số tốn hàng triệu đô la, nên thử mù không chấp nhận được. Điều ngành cần là một *luật cho phép ngoại suy*: chạy vài mô hình nhỏ, khớp một đường cong, rồi dự đoán mô hình lớn trước khi tiêu tiền.

Học xong mục này thì làm được gì? Ba việc mà trước đó bạn chỉ làm được bằng cách hỏi người khác. Một, tính chi phí huấn luyện của một mô hình bất kỳ bằng một phép nhân trên giấy. Hai, phát hiện khi một bảng kết quả đang so hai mô hình có ngân sách khác nhau. Ba, thiết kế thí nghiệm điểm giao trả lời đúng câu hỏi bạn đang có.

TRỰC GIÁC

Hãy mượn ngôn ngữ của **hiệu chỉnh chùm tia** (*bundle adjustment*), với ý thức rằng đây là phép loại suy chứ không phải định lý. Trong BA, số ảnh là toạ độ điểm và tư thế camera; số phương trình là số quan sát. Nếu bạn thêm điểm 3D mà không thêm quan sát, hệ trở nên thiếu ràng buộc: nghiệm vẫn tồn tại, sai số tái chiếu vẫn nhỏ, nhưng cấu trúc thu được là ảo — nó khớp quan sát mà không tương ứng với thế giới.

Bài học Chinchilla nói rằng cả ngành đã chạy đúng tình huống đó suốt vài năm: mô hình được thổi phồng số tham số trong khi số “quan sát” — *token* dữ liệu — gần như đứng yên. Loss huấn luyện vẫn giảm, bảng kết quả vẫn đẹp, nên không ai thấy có gì sai. Chỗ mà phép loại suy gây cũng đáng nhớ. Mạng nơ-ron quá tham số hoá vẫn tổng quát được, nên “thiếu ràng buộc” ở đây không sinh nghiệm vô nghĩa như trong BA. Nó chỉ sinh ra *lãng phí*: cùng số FLOPs ấy lẽ ra mua được mô hình tốt hơn.

B. Scaling laws — từ thử mù sang dự đoán (Scaling Laws)

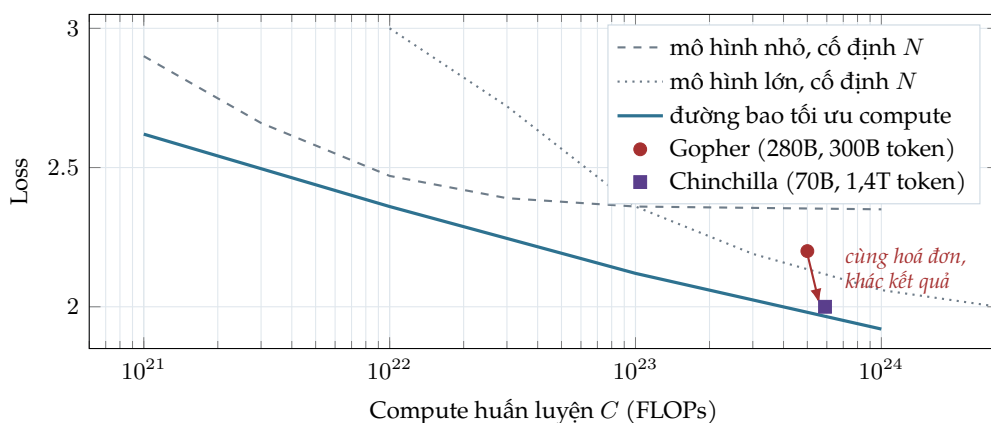
Quan sát thực nghiệm nền tảng là: khi tăng N , D và C , loss giảm theo quan hệ lũy thừa (*power law*) chứ không theo bậc thang [1]. Dạng thường dùng là

$$L(N, D) = L_{\infty} + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}}.$$

Công thức này thực sự nói điều gì? Rằng loss của bạn là tổng của ba thứ độc lập. Thứ nhất là phần sàn không bao giờ giảm được, L_{∞} : entropy thật của dữ liệu, cộng mọi thứ mà kiến trúc này về nguyên tắc không biểu diễn nổi. Thứ hai là phần phạt vì mô hình quá nhỏ. Thứ ba là phần phạt vì dữ liệu quá ít. Muốn giảm loss, bạn phải biết mình đang bị phạt ở số hạng nào — và đó chính là điều mà đường cong cho bạn biết.

Điều làm công thức có giá trị thực dụng không phải hình dạng của nó, mà là việc nó *khớp được từ các mô hình nhỏ và ngoại suy đúng lên vài bậc độ lớn*. Đó là lần đầu tiên trong lịch sử ngành, một quyết định chi tiêu chín chữ số được đưa ra dựa trên đường cong khớp từ thí nghiệm rẻ tiền. Hiện tượng tương tự đã được xác nhận cho thị giác với ViT [2], dù các hằng số hoàn toàn khác.

Hình 1 cho thấy vì sao lập luận này không nhìn ra được nếu chỉ theo dõi một mô hình: mỗi kích thước mô hình có đường loss riêng, và điều đáng quan tâm là *đường bao* của cả họ chứ không phải bất kỳ đường nào trong đó.



Hình 1. Đường bao tối ưu compute và hai mô hình cùng ngân sách (trục và hình dạng minh hoạ; chỉ vị trí tương đối của Gopher với Chinchilla là dựa trên kết quả đã công bố). Một mô hình cố định kích thước sẽ rời khỏi đường bao khi ta đổ thêm dữ liệu vào nó: loss chững lại vì nó đã hết chỗ chứa. Gopher nằm phía trên đường bao ở đúng mức compute của Chinchilla — đó là hình ảnh của việc thừa tham số và thiếu dữ liệu.

↔ **đối lập với** B/11 — chương kia đo giới hạn vật lý của phần cứng, mục này đo giới hạn kinh tế của việc dùng phần cứng đó — cùng một ngân sách nhìn từ hai phía.

Hai hệ quả cần đọc kỹ:

- **Lợi ích giảm dần nhưng không bao giờ dừng.** Luật lũy thừa không có ngưỡng mà thêm dữ liệu trở nên vô ích — chỉ có ngưỡng mà nó không còn đáng tiền. Câu hỏi đúng luôn là “đáng bao nhiêu”, không phải “còn tác dụng không”.
- **Scaling không sửa được lỗi bài toán.** Số hạng L_∞ nói rằng nếu nhân bản hoặc bài toán bị đặt sai, mọi đường cong đều chạm sàn ở một chỗ cao và không lượng compute nào cứu được.

C. Bài học Chinchilla (The Chinchilla Lesson)

Câu hỏi tiếp theo: với C cho trước, chọn N và D thế nào? Xấp xỉ chuẩn cho **transformer** là

$$C \approx 6ND \quad (\text{FLOPs cho toàn bộ quá trình huấn luyện}),$$

với hệ số 6 đến từ hai FLOPs cho mỗi phép nhân-cộng, nhân với ba lượt (một **lượt xuôi** (*forward pass*), hai **lượt ngược** (*backward pass*)). Diễn giải bằng lời: chi phí huấn luyện tỉ lệ thuận với tích của kích thước mô hình và lượng dữ liệu — nên với ngân sách cố định, hai đại lượng ấy nằm trên một đường hyperbol và bạn chỉ được chọn một điểm trên đó. Đáp số thực nghiệm là N và D nên tăng **cùng tốc độ**, mỗi cái tỉ lệ với $\sqrt{C}$ [3].

Ví dụ nhỏ nhất, dùng hai mô hình đã công bố.

- **Gopher:** $N = 280$ tỉ tham số, $D = 300$ tỉ **token** $\Rightarrow C = 6 \times 2,8 \times 10^{11} \times 3 \times 10^{11} \approx 5,0 \times 10^{23}$ FLOPs. Tỉ lệ $D/N = 1,07$.
- **Chinchilla:** $N = 70$ tỉ, $D = 1400$ tỉ **token** $\Rightarrow C = 6 \times 7 \times 10^{10} \times 1,4 \times 10^{12} \approx 5,9 \times 10^{23}$ FLOPs. Tỉ lệ $D/N = 20$.

Cùng bậc compute, tức cùng hoá đơn. Mô hình nhỏ hơn bốn lần nhưng ăn nhiều dữ liệu hơn gần năm lần thẳng rõ rệt trên hầu hết bài kiểm tra. Con số 20 **token** cho mỗi tham số chính là quy tắc ngón tay cái người ta nhớ.

Cơ chế của cái sai đáng học hơn con số 20. Số tham số là đại lượng dễ công bố, dễ so sánh và dễ gây ấn tượng; lượng **token** thì không ai khoe. Khi một đại lượng dễ đo trở thành đại lượng được tối ưu, nó thôi là đại lượng đúng — cùng một cơ chế đã làm hỏng nhiều bảng xếp hạng ở

E/24-thi-nghiem-va-ablation. Lưu ý thêm: con số 20 chỉ tối ưu cho *chi phí huấn luyện*. Mô hình phục vụ hàng tỉ lượt **suy luận** (*inference*) thì hoá đơn nằm ở phía chạy chứ không phải phía huấn luyện. Tối ưu tổng chi phí vòng đời vì thế đẩy bạn về mô hình *nhỏ hơn nữa*, huấn luyện lâu hơn nữa. Đó là lý do các mô hình mở gần đây thường vượt xa tỉ lệ 20.

D. Chất lượng hơn số lượng (Data Quality over Quantity)

Bước tiếp theo của cùng lập luận: nếu D quan trọng đến thế thì D là gì càng quan trọng hơn. Một dữ liệu bị lặp mười lần không mang thêm thông tin nào nhưng vẫn ngốn đúng số FLOPs như dữ liệu mới. Với thị giác, hiệu ứng còn mạnh hơn vì ảnh web trùng lặp ở mức độ khủng khiếp: cùng một logo, cùng một ảnh *stock*, cùng một khung hình phim xuất hiện hàng nghìn lần.

Điều này gắn thẳng vào B/06-chat-luong-du-lieu: các tập dữ liệu chuẩn của ngành có tỉ lệ nhân sai không nhỏ, đủ để đảo thứ hạng giữa các mô hình gần nhau [4]. Khi bạn tự tuyển *corpus*, một giờ viết bộ lọc trùng lặp thường có lãi cao hơn một giờ GPU.

E. Nhiễm bẩn tập kiểm tra — vấn đề do chính pretraining sinh ra

Tiền huấn luyện quy mô web tạo ra một vấn đề mà các phương pháp cũ không có: **nhiễm bẩn tập kiểm tra** (*test set contamination*). Nếu ảnh hoặc câu hỏi trong tập test đã nằm đâu đó trong *corpus* tiền huấn luyện, con số đánh giá không đo khả năng tổng quát hoá mà đo khả năng ghi nhớ. Cái làm nó nguy hiểm không phải bản thân hiện tượng — ai cũng biết là xấu — mà là việc nó *không kiểm chứng được*: *corpus* thường không công bố, và ngay cả khi công bố thì đối sánh hàng tỉ ảnh với tập test cũng không rẻ.

Có một dạng tinh vi hơn và phổ biến hơn: nhiễm bẩn qua cộng đồng. Một tập test công khai được cả ngành dùng mười năm sẽ bị việc chọn siêu tham số bào mòn dần, ngay cả khi không ai nhìn trực tiếp vào nó. Khi người ta dựng lại một tập test mới theo đúng quy trình cũ, hiệu năng mọi mô hình đều tụt, và tụt gần như song song [5]. Tính song song đó là tin tốt — thứ hạng tương đối còn dùng được — nhưng giá trị tuyệt đối thì không.

Kết luận thực dụng: với bất kỳ mô hình nền tảng nào, con số *zero-shot* trên *benchmark* công khai nên coi là chặn trên lạc quan. Bằng chứng duy nhất đáng tin là tập đánh giá do chính bạn thu, *sau* ngày mô hình được huấn luyện, trên đúng phân phối bạn sẽ triển khai.

→ **tiền đề** B/07 — quy trình chia dữ liệu sạch là điều kiện để con số đánh giá có nghĩa; nhiễm bẩn chỉ là rò rỉ dữ liệu ở quy mô web, nơi bạn không kiểm toán được nguồn.

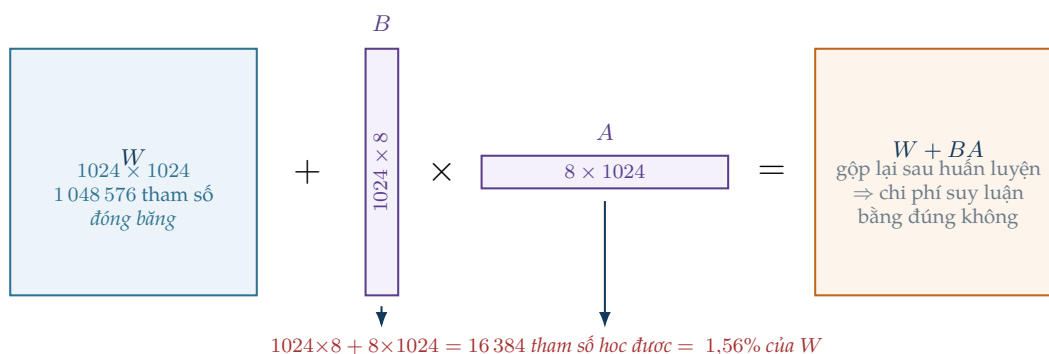
F. PEFT — đổi lại kinh tế học của việc thích nghi

Vấn đề rất cụ thể: tinh chỉnh toàn bộ một mô hình lớn nghĩa là lưu một bản sao toàn bộ trọng số cho mỗi khách hàng, mỗi tác vụ, mỗi phiên bản — và trạng thái *optimizer* của Adam còn tốn gấp đôi số đó. **PEFT** (tinh chỉnh chỉ một phần rất nhỏ tham số) giải bài toán này bằng cách đóng băng trọng số gốc và chỉ học một phần rất nhỏ tham số thêm vào:

- **Adapter** — chèn các khối nhỏ giữa các tầng, chỉ chúng được học [6].
- **LoRA** — phổ biến hơn: viết cập nhật dưới dạng tích hai ma trận hạng thấp, $\Delta W = BA$ với $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, $r \ll \min(d, k)$ [7]. *Ý nghĩa*: ta giả định rằng “khoảng cách” giữa mô hình gốc và mô hình bạn cần nằm trong một không gian con rất thấp chiều, nên chỉ cần học toạ độ trong không gian con đó.
- **Prompt tuning** — không đụng trọng số nào, chỉ học một chuỗi *token* đầu vào. Rẻ nhất, cũng yếu nhất.

Con số làm rõ mọi thứ. Một ma trận trọng số 1024×1024 có 1 048 576 tham số. LoRA hạng $r = 8$ thay nó bằng $1024 \times 8 + 8 \times 1024 = 16\,384$ tham số, tức 1,56%. Nhân lên toàn mô hình: một *adapter* vài chục megabyte thay cho một *checkpoint* vài chục gigabyte — và vì ΔW cộng thẳng vào W được sau khi huấn luyện, chi phí suy luận bằng đúng không.

Hình 2 vẽ cùng phép tính đó dưới dạng diện tích, vì đó là cách nhanh nhất để thấy vì sao con số 1,56% không phải một thủ thuật mà là hệ quả hình học của việc chọn $r \ll d$.



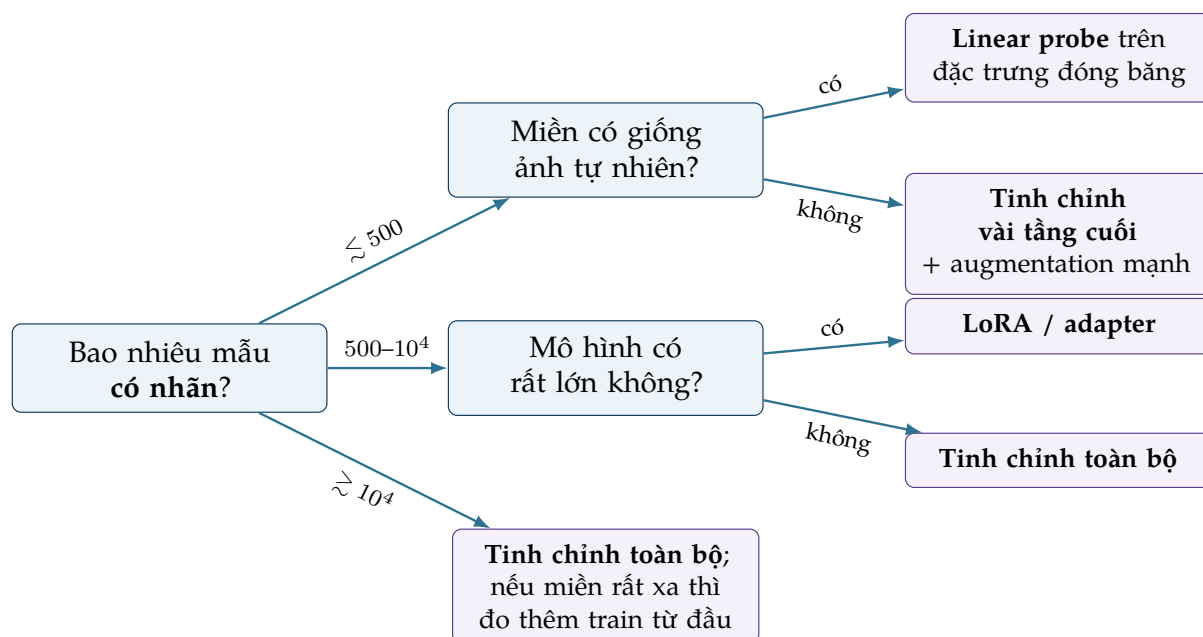
Hình 2. LoRA thay cập nhật đầy đủ ΔW bằng tích hai ma trận hạng thấp. Diện tích hai dải mỏng so với hình vuông lớn chính là tỉ lệ tham số phải học và phải lưu; và vì BA cộng thẳng vào W sau khi huấn luyện nên phần tiết kiệm này không phải trả lại ở lúc suy luận. Giả định ngầm là cập nhật cần thiết thật sự nằm trong một không gian con r chiều.

Giả định ngầm cần gọi tên: LoRA tin rằng *cập nhật cần thiết để thích nghi có hạng thấp*. Điều này hợp lý khi tác vụ đích là biến thể của thứ mô hình đã biết. Nó sai khi bạn đòi mô hình học một năng lực thực sự mới; khi đó hạng thấp thành cổ chai, và triệu chứng là loss huấn luyện dừng ở mức cao hơn hẳn tinh chỉnh toàn bộ bất kể học bao lâu. Phản ứng đúng lúc đó là tăng r hoặc mở thêm module, không phải tăng *learning rate*.

↔ cùng nguyên lý với $D/20$ — nén mô hình cũng giả định cái cần giữ nằm trong một không gian con nhỏ; LoRA là phiên bản của giả định đó ở phía thích nghi thay vì ở phía suy luận.

G. Quyết định cuối cùng vẫn là của bạn: thí nghiệm điểm giao

Mọi thứ ở trên là kiến thức chung. Hình 3 nén nó thành một cây quyết định dùng được trong ba mươi giây, và Bảng 1 nói rõ từng nhánh hổng ở đâu. Cả hai chỉ thu hẹp không gian tìm kiếm — chúng không kết luận thay bạn.

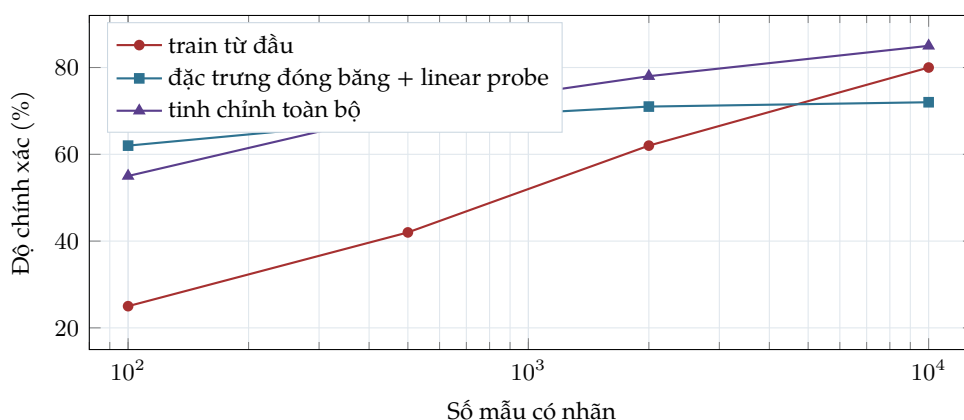


Hình 3. Cây quyết định chọn chế độ thích nghi. Các ngưỡng là điểm khởi đầu chứ không phải hằng số phổ quát — thí nghiệm điểm giao ở Hình 4 là cách xác định ngưỡng thật cho bài toán của bạn.

Bảng 1. Năm chế độ thích nghi, xếp theo số tham số được phép thay đổi.

Chế độ	Cơ chế	Giả định	Hỏng khi nào
Train từ đầu	Học mọi thứ từ dữ liệu của bạn	Dữ liệu đủ để học lại cả thống kê ảnh cơ bản	Dưới vài chục nghìn mẫu: gần như luôn thua
Linear probe	Chỉ học một siêu phẳng trên đặc trưng đóng băng	Thông tin cần thiết đã tuyến tính tách được	Tác vụ cần định vị chính xác, hoặc đặc trưng không có sẵn trong nguồn
Tinh chỉnh vài tầng cuối	Mở phần chuyên biệt của nguồn	Khác biệt miền nằm ở tầng cao	Khác biệt nằm ở tầng thấp (ảnh nhiệt, radar, đa phổ)
LoRA / adapter	Học cập nhật hạng thấp, gốc đóng băng	Cập nhật cần thiết có hạng thấp	Cần năng lực thực sự mới: loss chẳng ở mức cao
Tinh chỉnh toàn bộ	Mở toàn bộ	Đủ dữ liệu để không quên đặc trưng gốc	Dữ liệu ít: overfit và quên thảm khốc

Thí nghiệm đáng làm nhất của cả chương là thí nghiệm điểm giao: chạy ba chế độ — train từ đầu, đóng băng, tinh chỉnh toàn bộ — ở các mức 100, 500, 2000, 10 000 mẫu, mỗi mức lặp ba *seed*, rồi vẽ ba đường cong trên một hình. Hình 4 minh họa hình dạng thường thấy, với cảnh báo rõ ràng rằng các con số là *minh họa*, không phải kết quả đo của một tập dữ liệu cụ thể.



Hình 4. Hình dạng điển hình của thí nghiệm điểm giao (số liệu minh họa, không phải kết quả đo). Đường đóng băng bảo hòa sớm nhưng thắng ở vùng dữ liệu rất ít; tinh chỉnh toàn bộ vượt lên sau điểm giao thứ nhất; train từ đầu chỉ cạnh tranh ở vùng dữ liệu lớn. Vị trí hai điểm giao khác nhau cho từng bài toán — đó chính là thứ bạn phải đo.

Hai điểm giao trên hình là hai con số đáng giá hơn mọi lời khuyên chung, vì chúng trả lời đúng câu hỏi bạn thực sự có: *với lượng nhãn tôi đang có, và với lượng nhãn tôi mua thêm được trong quý này, chế độ nào đúng?* Và vì trục hoành là log, chỉ bốn điểm đo đã đủ để nhìn ra hình dạng.

H. Giới hạn và trường hợp hỏng

- **Scaling law là quan hệ khớp thực nghiệm, không phải định luật.** Nó chỉ đúng *trong công thức huấn luyện đã dùng để khớp nó*: đổi kiến trúc, đổi lịch *learning rate*, đổi phân bố dữ liệu là các hằng số thay đổi và đường ngoại suy trượt. Đã có ít nhất một lần cả ngành ngoại suy sai vì công thức khớp có một chi tiết lịch học không nhất quán — chính là câu chuyện Chinchilla.
- **Scaling law mô tả loss, không mô tả năng lực.** Loss giảm mượt trong khi các năng lực rời rạc (đếm, suy luận nhiều bước) xuất hiện không mượt, nên đường cong đẹp không cho phép hứa hẹn gì về hành vi ở tác vụ đích.
- **LoRA dễ bị dùng để né việc chẩn đoán.** Khi kết quả kém, người ta tăng hạng thay vì hỏi liệu bài toán có thực sự là bài toán thích nghi hay không.
- **So sánh chỉ có nghĩa khi công sức tune ngang nhau.** So một *baseline* chưa tune với phương pháp đã tune là dạng thí nghiệm phổ biến nhất và vô giá trị nhất trong ngành [8].

CẠM BÃY

Ba lỗi hay đi cùng nhau khi chạy thí nghiệm điểm giao. **Một:** dùng cùng một tập *validation* để chọn siêu tham số cho cả ba chế độ rồi báo cáo trên chính tập đó — điểm giao dịch chuyển theo hướng có lợi cho chế độ được tune nhiều nhất. **Hai:** chạy một *seed* mỗi điểm; với 100 mẫu, độ lệch giữa các *seed* thường lớn hơn khoảng cách giữa hai chế độ, nên đường cong bạn vẽ là nhiễu. **Ba:** quên rằng *backbone* đã tiền huấn luyện có thể đã nhìn thấy dữ liệu đích của bạn — nếu ảnh của bạn từng ở trên internet, “*zero-shot*” không còn là *zero-shot*.

CẦN NHỚ

Compute, tham số và dữ liệu là ba mặt của cùng một ngân sách; tối ưu một mặt mà không nhìn hai mặt kia là cách sai tiền có hệ thống. Nhớ kèm hai công thức: $C \approx 6ND$ để ước lượng hoá đơn, và tỉ lệ $D/N \approx 20$ như điểm khởi đầu (không phải chân lý). Và mọi quy

tắc chung chỉ là điểm khởi đầu — con số duy nhất ràng buộc quyết định của bạn là điểm giao đo trên chính dữ liệu của bạn. [T2]

I. Kết nối

Mục này là mặt định lượng của C/18/01-nguồn-giam-sat: mục kia nói giám sát lấy ở đâu, mục này nói mua bao nhiêu thì đủ. Nó là tiền đề của D/20-toi-uu-va-nen-mo-hinh, vì phần lớn kỹ thuật nén chỉ là cách mua lại compute suy luận bằng compute huấn luyện — cùng một logic ngân sách, chỉ khác cột chi. Nó phụ thuộc rất chặt vào B/07-chia-du-lieu-va-danh-gia: không có quy trình chia dữ liệu sạch thì nhiệm bản biến mọi con số ở đây thành vô nghĩa. Và nó đối lập có ích với B/11-gioi-han-tinh-toan: chương kia đo giới hạn vật lý của phần cứng, mục này đo giới hạn kinh tế của việc dùng phần cứng đó.

I Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao Gopher và Chinchilla tốn xấp xỉ cùng số FLOPs, và điều gì trong cách ngành đo lường tiến bộ đã khiến sai lầm này sống sót nhiều năm?
2. Tỷ lệ 20 *token* trên tham số tối ưu cho chi phí gì? Nếu mô hình sẽ phục vụ một tỉ lượt suy luận thì tỉ lệ đúng dịch về phía nào, và vì sao?
3. Số hạng L_∞ nói gì về việc dùng compute để sửa một bài toán bị đặt sai?
4. LoRA hạng 8 trên ma trận 1024×1024 tiết kiệm bao nhiêu phần trăm tham số, và triệu chứng nào cho biết giả định hạng thấp đã bị vi phạm?
5. Vì sao khi dựng lại một tập test theo đúng quy trình cũ thì mọi mô hình đều tụt điểm, và vì sao việc chúng tụt gần như song song lại là tin tốt?
6. Ba cách phổ biến làm cho thí nghiệm điểm giao ra kết quả sai là gì, và sửa từng cái thế nào?

Nguồn

- [1] J. Kaplan, S. McCandlish, T. Henighan, et al., “Scaling Laws for Neural Language Models,” *arXiv:2001.08361*, 2020.
- [2] X. Zhai, A. Kolesnikov, N. Houlsby, et al., “Scaling Vision Transformers,” in *CVPR*, 2022.
- [3] J. Hoffmann, S. Borgeaud, A. Mensch, et al., “Training Compute-Optimal Large Language Models,” in *NeurIPS*, 2022.
- [4] C. G. Northcutt, A. Athalye, and J. Mueller, “Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [5] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [6] N. Houlsby, A. Giurgiu, S. Jastrzebski, et al., “Parameter-Efficient Transfer Learning for NLP,” in *ICML*, 2019.
- [7] E. J. Hu, Y. Shen, P. Wallis, et al., “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [8] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, “Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches,” in *RecSys*, 2019.